

Chapter 18

PetitParser: Building Modular Parsers

with the participation of:

Jan Kurs (kurs@iam.unibe.ch)

Guillaume Larcheveque (guillaume.larcheveque@gmail.com)

Lukas Renggli (renggli@gmail.com)

Building parsers to analyze and transform data is a common task in software development. In this chapter we present a powerful parser framework called PetitParser. PetitParser combines many ideas from various parsing technologies to model grammars and parsers as objects that can be reconfigured dynamically. PetitParser was written by Lukas Renggli as part of his work on the Helvetia system ¹ but it can be used as a standalone library.

18.1 Writing parsers with PetitParser

PetitParser is a parsing framework different from many other popular parser generators. PetitParser makes it easy to define parsers with Smalltalk code and to dynamically reuse, compose, transform and extend grammars. We can reflect on the resulting grammars and modify them on-the-fly. As such PetitParser fits better the dynamic nature of Smalltalk.

Furthermore, PetitParser is not based on tables such as SmaCC and ANTLR. Instead it uses a combination of four alternative parser methodologies: scannerless parsers, parser combinators, parsing expression grammars and packrat parsers. As such PetitParser is more powerful in what it can

¹<http://scg.unibe.ch/research/helvetia>

parse. Let's have a quick look at these four parser methodologies:

Scannerless Parsers combine what is usually done by two independent tools (scanner and parser) into one. This makes writing a grammar much simpler and avoids common problems when grammars are composed.

Parser Combinators are building blocks for parsers modeled as a graph of composable objects; they are modular and maintainable, and can be changed, recomposed, transformed and reflected upon.

Parsing Expression Grammars (PEGs) provide the notion of ordered choices. Unlike parser combinators, the ordered choice of PEGs always follows the first matching alternative and ignores other alternatives. Valid input always results in exactly one parse-tree, the result of a parse is never ambiguous.

Packrat Parsers give linear parse-time guarantees and avoid common problems with left-recursion in PEGs.

Loading PetitParser

Enough talking, let's get started. PetitParser is developed in Pharo, and there are also versions for Java and Dart available. A ready made image can be downloaded². To load PetitParser into an existing image evaluate the following Gofer expression:

Script 18.1: *Installing PetitParser*

```
Gofer new
  smalltalkhubUser: 'Moose' project: 'PetitParser';
  package: 'ConfigurationOfPetitParser';
  load.
(Smalltalk at: #ConfigurationOfPetitParser) perform: #loadDefault.
```

More information on how to get PetitParser can be found on the chapter about petit parser in the Moose book.³

Writing a simple grammar

Writing grammars with PetitParser is as simple as writing Smalltalk code. For example, to define a grammar that parses identifiers starting with a letter followed by zero or more letters or digits is defined and used as follows:

²<https://ci.inria.fr/moose/job/petitparser/>

³<http://www.themoosebook.org/book/internals/petit-parser>



Figure 18.1: Syntax diagram representation for the identifier parser defined in script 18.2

Script 18.2: *Creating our first parser to parse identifiers*

```

|identifier|
identifier := #letter asParser , #word asParser star.
identifier parse: 'a987jlkj' → #($a #($9 $8 $7 $j $l $k $j))

```

A graphical notation

Figure 18.1 presents a syntax diagram of the identifier parser. Each box represents a parser. The arrows between the boxes represent the flow in which input is consumed. The rounded boxes are elementary parsers (terminals). The squared boxes (not shown on this figure) are parsers composed of other parsers (non terminals).

If you inspect the object `identifier` of the previous script, you'll notice that it is an instance of a `PPSequenceParser`. If you dive further into the object you will notice the following tree of different parser objects:

Script 18.3: *Composition of parsers used for the identifier parser*

```

PPSequenceParser (accepts a sequence of parsers)
  PPPredicateObjectParser (accepts a single letter)
  PPPossessiveRepeatingParser (accepts zero or more instances of another parser)
    PPPredicateObjectParser (accepts a single word character)

```

The root parser is a sequence parser because the `,` (comma) operator creates a sequence of (1) a letter parser and (2) zero or more word character parser. The root parser first child is a predicate object parser created by the `#letter asParser` expression. This parser is capable of parsing a single letter as defined by the `Character»isLetter` method. The second child is a repeating parser created by the `star` call. This parser uses its child parser (another predicate object parser) as much as possible on the input (*i.e.*, it is a *greedy* parser). Its child parser is a predicate object parser created by the `#word asParser` expression. This parser is capable of parsing a single digit or letter as defined by the `Character»isDigit` and `Character»isLetter` methods.

Parsing some input

To actually parse a string (or stream) we use the method `PPParser»parse:` as follows:

Script 18.4: *Parsing some input strings with the identifier parser*

```
identifier parse: 'yeah'.      —→  #($y #($e $a $h))
identifier parse: 'f123'.     —→  #($f #($1 $2 $3))
```

While it seems odd to get these nested arrays with characters as a return value, this is the default decomposition of the input into a parse tree. We'll see in a while how that can be customized.

If we try to parse something invalid we get an instance of `PPFailure` as an answer:

Script 18.5: *Parsing invalid input results in a failure*

```
identifier parse: '123'.      —→  letter expected at 0
```

This parsing results in a failure because the first character (1) is not a letter. Instances of `PPFailure` are the only objects in the system that answer with `true` when you send the message `#isPetitFailure`. Alternatively you can also use `PPParser»parse: onError:` to throw an exception in case of an error:

```
identifier
  parse: '123'
  onError: [ :msg :pos | self error: msg ].
```

If you are only interested if a given string (or stream) matches or not you can use the following constructs:

Script 18.6: *Checking that some inputs are identifiers*

```
identifier matches: 'foo'.      —→  true
identifier matches: '123'.     —→  false
identifier matches: 'foo()'.   —→  true
```

The last result can be surprising: indeed, a parenthesis is neither a digit nor a letter as was specified by the `#word asParser` expression. In fact, the identifier parser matches “foo” and this is enough for the `PPParser»matches:` call to return `true`. The result would be similar with the use of `parse:` which would return `#($f #($o $o))`.

If you want to be sure that the complete input is matched, use the message `PPParser»end` as follows:

Script 18.7: *Ensuring that the whole input is matched using `PPParser»end`*

```
identifier end matches: 'foo()'. —→  false
```

The `PPParser»end` message creates a new parser that matches the end of input. To be able to compose parsers easily, it is important that parsers do not match the end of input by default. Because of this, you might be interested to find all the places that a parser can match using the message `PPParser»matchesSkipIn:` and `PPParser»matchesIn:`.

Script 18.8: *Finding all matches in an input*

```
identifier matchesSkipIn: 'foo 123 bar12'.
→ an OrderedCollection(#($f #($o $o)) #($b #($a $r $1 $2)))

identifier matchesIn: 'foo 123 bar12'.
→ an OrderedCollection(#($f #($o $o)) #($o #($o)) #($o #()) #($b #($a $r $1 $2))
  #($a #($r $1 $2)) #($r #($1 $2)))
```

The `PPParser»matchesSkipIn:` method returns a collection of arrays containing what has been matched. This function avoids parsing the same character twice. The method `PPParser»matchesIn:` does a similar job but returns a collection with all possible sub-parsed elements: *e.g.*, evaluating `identifier matchesIn: 'foo 123 bar12'` returns a collection of 6 elements.

Similarly, to find all the matching ranges (index of first character and index of last character) in the given input one can use either `PPParser»matchingSkipRangesIn:` or `PPParser»matchingRangesIn:` as shown by the script below:

Script 18.9: *Finding all matched ranges in an input*

```
identifier matchingSkipRangesIn: 'foo 123 bar12'.
→ an OrderedCollection((1 to: 3) (9 to: 13))

identifier matchingRangesIn: 'foo 123 bar12'.
→ an OrderedCollection((1 to: 3) (2 to: 3) (3 to: 3) (9 to: 13) (10 to: 13) (11 to: 13))
```

Different kinds of parsers

PetitParser provide a large set of ready-made parser that you can compose to consume and transform arbitrarily complex languages. The terminal parsers are the most simple ones. We've already seen a few of those, some more are defined in the protocol Table 18.1.

The class side of `PPPredicateObjectParser` provides a lot of other factory methods that can be used to build more complex terminal parsers. To use them, send the message `PPParser»asParser` to a symbol containing the name of the factory method (such as `#punctuation asParser`).

The next set of parsers are used to combine other parsers together and is defined in the protocol Table 18.2.

Terminal Parsers	Description
\$a asParser	Parses the character \$a.
'abc' asParser	Parses the string 'abc'.
#any asParser	Parses any character.
#digit asParser	Parses one digit (0..9).
#letter asParser	Parses one letter (a..z and A..Z).
#word asParser	Parses a digit or letter.
#blank asParser	Parses a space or a tabulation.
#newline asParser	Parses the carriage return or line feed characters.
#space asParser	Parses any white space character including new line.
#tab asParser	Parses a tab character.
#lowercase asParser	Parses a lowercase character.
#uppercase asParser	Parses an uppercase character.
nil asParser	Parses nothing.

Table 18.1: PetitParser pre-defines a multitude of terminal parsers

Parser Combinators	Description
p1 , p2	Parses p1 followed by p2 (sequence).
p1 / p2	Parses p1, if that doesn't work parses p2.
p star	Parses zero or more p.
p plus	Parses one or more p.
p optional	Parses p if possible.
p and	Parses p but does not consume its input.
p negate	Parses p and succeeds when p fails.
p not	Parses p and succeeds when p fails, but does not consume its input.
p end	Parses p and succeeds only at the end of the input.
p times: n	Parses p exactly n times.
p min: n max: m	Parses p at least n times up to m times
p starLazy: q	Like star but stop consuming when q succeeds

Table 18.2: PetitParser pre-defines a multitude of parser combinators

As a simple example of parser combination, the following definition of the identifier2 parser is equivalent to our previous definition of identifier:

Script 18.10: *A different way to express the identifier parser*

```
identifier2 := #letter asParser , (#letter asParser / #digit asParser) star.
```

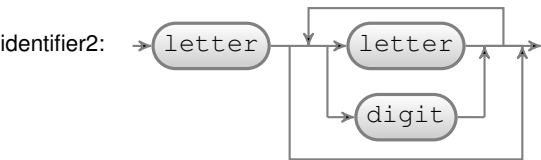


Figure 18.2: Syntax diagram representation for the identifier2 parser defined in script 18.10

Parser action

To define an action or transformation on a parser we can use one of the messages `PPParser>==>`, `PPParser>flatten`, `PPParser>token` and `PPParser>trim` defined in the protocol Table 18.3.

Action Parsers	Description
p flatten	Creates a string from the result of p.
p token	Similar to flatten but returns a PPToken with details.
p trim	Trims white spaces before and after p.
p trim: trimParser	Trims whatever trimParser can parse (e.g., comments).
p ==> aBlock	Performs the transformation given in aBlock.

Table 18.3: PetitParser pre-defines a multitude of action parsers

To return a string of the parsed identifier instead of getting an array of matched elements, configure the parser by sending it the message `PPParser>flatten`.

Script 18.11: Using *flatten* so that the parsing result is a string

```
|identifier|
identifier := (#letter asParser , (#letter asParser / #digit asParser) star).
identifier parse: 'ajka0'          ->  #($a #($j $k $a $0))

identifier flatten parse: 'ajka0'  ->  'ajka0'
```

The message `PPParser>token` is similar to `flatten` but returns a `PPToken` that provide much more contextual information like the collection where the token was located and its position in the collection.

Sending the message `PPParser>trim` configures the parser to ignore white spaces at the beginning and end of the parsed result. In the following, using the first parser on the input leads to an error because the parser does not accept the spaces. With the second parser, spaces are ignored and removed from the result.



Figure 18.3: Syntax diagram representation for the number parser defined in script 18.14

Script 18.12: *Using PPParser»trim to ignore spaces*

```

|identifier|
identifier := (#letter asParser , #word asParser star) flatten.
identifier parse: ' ajka '      →    letter expected at 0

identifier trim parse: ' ajka '      →    'ajka'
  
```

Sending the message trim is equivalent to calling PPParser»trim: with #space asParser as a parameter. That means trim: can be useful to ignore other data from the input, source code comments for example:

Script 18.13: *Using PPParser»trim: to ignore comments*

```

| identifier comment ignorable line |
identifier := (#letter asParser , #word asParser star) flatten.
comment := '/' asParser, #newline asParser negate star.
ignorable := comment / #space asParser.
line := identifier trim: ignorable.
line parse: '// This is a comment
onIdentifier // another comment'      →    'onIdentifier'
  
```

The message PPParser»==> lets you specify a block to be executed when the parser matches an input. The next section presents several examples. Here is a simple way to get a number from its string representation.

Script 18.14: *Parsing integers*

```

number := #digit asParser plus flatten ==> [ :str | str asNumber ].
number parse: '123'      →    123
  
```

The table 18.3 shows the basic elements to build parsers. There are a few more well documented and tested factory methods in the operators protocols of PPParser. If you want to know more about these factory methods, browse these protocols. An interesting one is separatedBy: which answers a new parser that parses the input one or more times, with separations specified by another parser.

Writing a more complicated grammar

We now write a more complicated grammar for evaluating simple arithmetic expressions. With the grammar for a number (actually an integer) defined

above, the next step is to define the productions for addition and multiplication in order of precedence. Note that we instantiate the productions as PPDelegateParser upfront, because they recursively refer to each other. The method #setParser: then resolves this recursion. The following script defines three parsers for the addition, multiplication and parenthesis (see Figure 18.4 for the related syntax diagram):

Script 18.15: *Parsing arithmetic expressions*

```
term := PPDelegateParser new.
prod := PPDelegateParser new.
prim := PPDelegateParser new.

term setParser: (prod , $+ asParser trim , term ==> [ :nodes | nodes first + nodes last ])
/ prod.
prod setParser: (prim , $* asParser trim , prod ==> [ :nodes | nodes first * nodes last ])
/ prim.
prim setParser: ($ ( asParser trim , term , $) asParser trim ==> [ :nodes | nodes second ])
/ number.
```

The term parser is defined as being either (1) a prod followed by '+', followed by another term or (2) a prod. In case (1), an action block asks the parser to compute the arithmetic addition of the value of the first node (a prod) and the last node (a term). The prod parser is similar to the term parser. The prim parser is interesting in that it accepts left and right parenthesis before and after a term and has an action block that simply ignores them.

To understand the precedence of productions, see Figure 18.5. The root of the tree in this figure (term), is the production that is tried first. A term is either a + or a prod. The term production comes first because + as the lowest priority in mathematics.

To make sure that our parser consumes all input we wrap it with the end parser into the start production:

```
start := term end.
```

That's it, we can now test our parser:

Script 18.16: *Trying our arithmetic expressions evaluator*

```
start parse: '1 + 2 * 3'.    → 7
start parse: '(1 + 2) * 3'. → 9
```

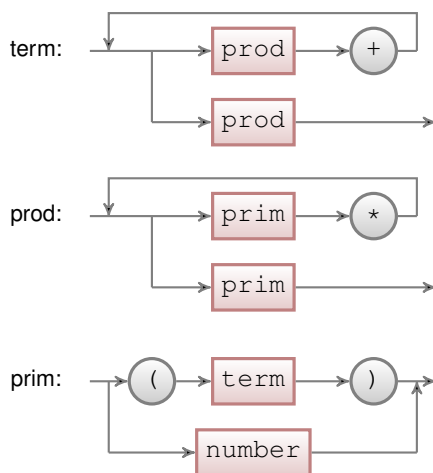


Figure 18.4: Syntax diagram representation for the term, prod, and prim parsers defined in script 18.15

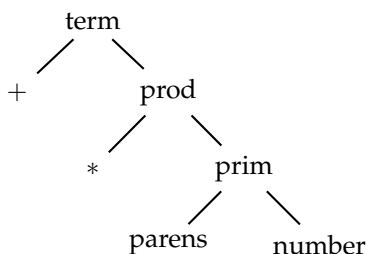


Figure 18.5: Explains how to understand the precedence of productions. An expression is a term which is either a sum or a production. It is necessary to recognize sums first as they have the lowest priority. A production is either a multiplication or a primitive. A primitive is either a parenthesised expression or a number.

18.2 Composite grammars with PetitParser

In the previous section we saw the basic principles of PetitParser and gave some introductory examples. In this section we are going to present a way to define more complicated grammars. We continue where we left off with the arithmetic expression grammar.

Writing parsers as a script as we did previously can be cumbersome, especially when grammar productions are mutually recursive and refer to each other in complicated ways. Furthermore a grammar specified in a sin-

gle script makes it unnecessary hard to reuse specific parts of that grammar. Luckily there is PPCompositeParser to the rescue.

Defining the grammar

As an example let's create a composite parser using the same expression grammar we built in the last section but this time we define it inside a class subclass of PPCompositeParser.

Script 18.17: *Creating a class to hold our arithmetic expression grammar*

```
PPCompositeParser subclass: #ExpressionGrammar
  instanceVariableNames: "
  classVariableNames: "
  poolDictionaries: "
  category: 'PetitTutorial'
```

Again we start with the grammar for an integer number. Define the method number as follows:

Script 18.18: *Implementing our first parser as a method*

```
ExpressionGrammar>>number
  ^ #digit asParser plus flatten trim ==> [ :str | str asNumber ]
```

Every production in ExpressionGrammar is specified as a method that returns its parser. Similarly, we define the productions term, prod, mul, and prim. Productions refer to each other by reading the respective instance variable of the same name and PetitParser takes care of initializing these instance variables for you automatically. We let Pharo automatically add the necessary instance variables as we refer to them for the first time. We obtain the following class definition:

Script 18.19: *Creating a class to hold our arithmetic expression grammar*

```
PPCompositeParser subclass: #ExpressionGrammar
  instanceVariableNames: 'add prod term mul prim parens number'
  classVariableNames: "
  poolDictionaries: "
  category: 'PetitTutorial'
```

Script 18.20: *Defining more expression grammar parsers, this time with no associated action*

```
ExpressionGrammar>>term
  ^ add / prod

ExpressionGrammar>>add
  ^ prod , $+ asParser trim , term
```

```

ExpressionGrammar>>prod
  ^ mul / prim

ExpressionGrammar>>mul
  ^ prim , $* asParser trim , prod

ExpressionGrammar>>prim
  ^ parens / number

ExpressionGrammar>>parens
  ^ $( asParser trim , term , $) asParser trim

```

Contrary to our previous implementation we do not define the production actions yet (what we previously did by using `PPParser»==>`); and we factor out the parts for addition (`add`), multiplication (`mul`), and parenthesis (`parens`) into separate productions. This will give us better reusability later on. For example, a subclass may override such methods to produce slightly different production output. Usually, production methods are categorized in a protocol named `grammar` (which can be refined into more specific protocol names when necessary such as `grammar—literals`).

Last but not least we define the starting point of the expression grammar. This is done by overriding `PPCompositeParser»start` in the `ExpressionGrammar` class:

Script 18.21: Defining the starting point of our expression grammar parser

```

ExpressionGrammar>>start
  ^ term end

```

Instantiating the `ExpressionGrammar` gives us an expression parser that returns a default abstract-syntax tree:

Script 18.22: Testing our parser on simple arithmetic expressions

```

parser := ExpressionGrammar new.
parser parse: '1 + 2 * 3'.      —→  #(1 $+ #(2 $* 3))
parser parse: '(1 + 2) * 3'.   —→  #(#($ ( #(1 $+ 2) $)) $* 3)

```

Writing dependent grammars

You can easily reuse parsers defined by other grammars. For example, imagine you want to create a new grammar that reuses the definition of `number` in the `ExpressionGrammar` we have just defined. For this, you have to declare a dependency to `ExpressionGrammar`:

Script 18.23: *Reusing the number parser from the ExpressionGrammar grammar*

```

PPCompositeParser subclass: #MyNewGrammar
  instanceVariableNames: 'number'
  classVariableNames: ''
  poolDictionaries: ''
  category: 'PetitTutorial'

MyNewGrammar class>>dependencies
  "Answer a collection of PPCompositeParser classes that this parser directly
  depends on."
  ^ {ExpressionGrammar}

MyNewGrammar>>number
  "Answer the same parser as ExpressionGrammar>>number."
  ^ (self dependencyAt: ExpressionGrammar) number

```

Defining an evaluator

Now that we have defined a grammar we can reuse this definition to implement an evaluator. To do this we create a *subclass* of ExpressionGrammar called ExpressionEvaluator.

Script 18.24: *Separating the grammar from the evaluator by creating a subclass*

```

ExpressionGrammar subclass: #ExpressionEvaluator
  instanceVariableNames: ''
  classVariableNames: ''
  poolDictionaries: ''
  category: 'PetitTutorial'

```

We then redefine the implementation of add, mul and parens with our evaluation semantics. This is accomplished by calling the super implementation and adapting the returned parser as shown in the following methods.

Script 18.25: *Refining the definition of some parsers to evaluate arithmetic expressions*

```

ExpressionEvaluator>>add
  ^ super add ==> [ :nodes | nodes first + nodes last ]

ExpressionEvaluator>>mul
  ^ super mul ==> [ :nodes | nodes first * nodes last ]

ExpressionEvaluator>>parens
  ^ super parens ==> [ :nodes | nodes second ]

```

The evaluator is now ready to be tested:

Script 18.26: Testing our evaluator on simple arithmetic expressions

```

parser := ExpressionEvaluator new.
parser parse: '1 + 2 * 3'.      → 7
parser parse: '(1 + 2) * 3'.    → 9

```

Defining a Pretty-Printer

We can reuse the grammar for example to define a simple pretty printer. This is as easy as subclassing ExpressionGrammar again!

Script 18.27: Separating the grammar from the pretty printer by creating a subclass

```

ExpressionGrammar subclass: #ExpressionPrinter
  instanceVariableNames: ""
  classVariableNames: ""
  poolDictionaries: ""
  category: 'PetitTutorial'

ExpressionPrinter>>add
  ^ super add ==> [:nodes | nodes first , ' + ' , nodes third]

ExpressionPrinter>>mul
  ^ super mul ==> [:nodes | nodes first , ' * ' , nodes third]

ExpressionPrinter>>number
  ^ super number ==> [:num | num printString]

ExpressionPrinter>>parens
  ^ super parens ==> [:node | '(' , node second , ')']

```

This pretty printer can be tried out as shown by the following expressions.

Script 18.28: Testing our pretty printer on simple arithmetic expressions

```

parser := ExpressionPrinter new.
parser parse: '1+2 *3'.      → '1 + 2 * 3'
parser parse: '(1 + 2 ) * 3'. → '(1 + 2) * 3'

```

Easy expressions with PPExpressionParser

PetitParser proposes a powerful tool to create expressions; PPExpressionParser is a parser to conveniently define an expression grammar with prefix, postfix, and left- and right-associative infix operators. The operator-groups are defined in descending precedence.

Script 18.29: *The ExpressionGrammar we previously defined can be implemented in few lines*

```
| expression parens number |
expression := PPEXpressionParser new.
parens := $( asParser token trim , expression , $) asParser token trim
==> [ :nodes | nodes second ].
number := #digit asParser plus flatten trim ==> [ :str | str asNumber ].

expression term: parens / number.

expression
group: [ :g |
  g left: $* asParser token trim do: [ :a :op :b | a * b ].
  g left: $/ asParser token trim do: [ :a :op :b | a / b ] ];
group: [ :g |
  g left: $+ asParser token trim do: [ :a :op :b | a + b ].
  g left: $- asParser token trim do: [ :a :op :b | a - b ] ].
```

Script 18.30: *Now our parser is also able to manage subtraction and division*

```
expression parse: '1-2/3'.      —>   (1/3)
```

18.3 Testing a grammar

The PetitParser contains a framework dedicated to testing your grammars. Testing a grammar is done by subclassing PPCompositeParserTest as follows:

Script 18.31: *Creating a class to hold the tests for our arithmetic expression grammar*

```
PPCompositeParserTest subclass: #ExpressionGrammarTest
  instanceVariableNames: ""
  classVariableNames: ""
  poolDictionaries: ""
  category: 'PetitTutorial'
```

It is then important that the test case class references the parser class: this is done by overriding the PPCompositeParserTest>>parserClass method in ExpressionGrammarTest:

Script 18.32: *Linking our test case class to our parser*

```
ExpressionGrammarTest>>parserClass
^ ExpressionGrammar
```

Writing a test scenario is done by implementing new methods in ExpressionGrammarTest:

Script 18.33: Implementing tests for our arithmetic expression grammar

```
ExpressionGrammarTest>>testNumber
self parse: '123 ' rule: #number.
```

```
ExpressionGrammarTest>>testAdd
self parse: '123+77' rule: #add.
```

These tests ensure that the ExpressionGrammar parser can parse some expressions using a specified production rule. Testing the evaluator and pretty printer is similarly easy:

Script 18.34: Testing the evaluator and pretty printer

```
ExpressionGrammarTest subclass: #ExpressionEvaluatorTest
instanceVariableNames: "
classVariableNames: "
poolDictionaries: "
category: 'PetitTutorial'
```

```
ExpressionEvaluatorTest>>parserClass
^ ExpressionEvaluator
```

```
ExpressionEvaluatorTest>>testAdd
super testAdd.
self assert: result equals: 200
```

```
ExpressionEvaluatorTest>>testNumber
super testNumber.
self assert: result equals: 123
```

```
ExpressionGrammarTest subclass: #ExpressionPrinterTest
instanceVariableNames: "
classVariableNames: "
poolDictionaries: "
category: 'PetitTutorial'
```

```
ExpressionPrinterTest>>parserClass
^ ExpressionPrinter
```

```
ExpressionPrinterTest>>testAdd
super testAdd.
self assert: result equals: '123 + 77'
```

```
ExpressionPrinterTest>>testNumber
super testNumber.
self assert: result equals: '123'
```


18.4 Case Study: A JSON Parser

In this section we illustrate PetitParser through the development of a JSON parser. JSON is a lightweight data-interchange format defined in <http://www.json.org>. We are going to use the specification on this website to define our own JSON parser.

JSON is a simple format based on nested pairs and arrays. The following script gives an example taken from Wikipedia <http://en.wikipedia.org/wiki/JSON>

Script 18.35: *An example of JSON*

```
{  "firstName": "John",
    "lastName": "Smith",
    "age": 25,
    "address":
      {  "streetAddress": "21 2nd Street",
          "city": "New York",
          "state": "NY",
          "postalCode": "10021" },
    "phoneNumber":
      [
        {  "type": "home",
            "number": "212 555-1234" },
        {  "type": "fax",
            "number": "646 555-4567" } ] }
```

JSON consists of object definitions (between curly braces “{}”) and arrays (between square brackets “[]”). An object definition is a set of key/value pairs whereas an array is a list of values. The previous JSON example then represents an object (a person) with several key/value pairs (*e.g.*, for the person’s first name, last name, and age). The address of the person is represented by another object while the phone number is represented by an array of objects.

First we define a grammar as subclass of `PPCompositeParser`. Let us call it `PPJsonGrammar`

Script 18.36: *Defining the JSON grammar class*

```
PPCompositeParser subclass: #PPJsonGrammar
instanceVariableNames: ""
classVariableNames: 'CharacterTable'
poolDictionaries: ""
category: 'PetitJson-Core'
```

We define the `CharacterTable` class variable since we will later use it to parse strings.

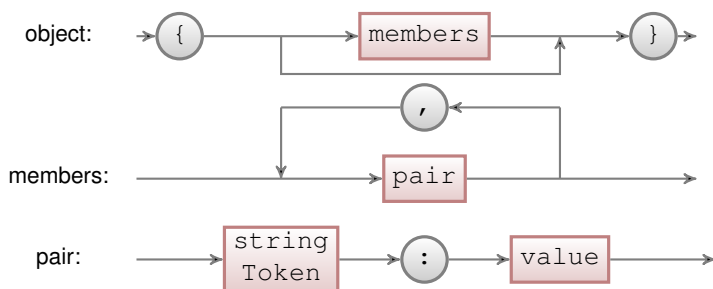


Figure 18.6: Syntax diagram representation for the JSON object parser defined in script 18.37

Parsing objects and arrays

The syntax diagrams for JSON objects and arrays are in Figure 18.6 and Figure 18.7. A PetitParser can be defined for JSON objects with the following code:

Script 18.37: *Defining the JSON parser for object as represented in Figure 18.6*

```

PPJsonGrammar>>object
^ ${ asParser token trim , members optional , $} asParser token trim

PPJsonGrammar>>members
^ pair separatedBy: $, asParser token trim

PPJsonGrammar>>pair
^ stringToken , $: asParser token trim , value
  
```

The only new thing here is the call to the `PPParser»separatedBy:` convenience method which answers a new parser that parses the receiver (a value here) one or more times, separated by its parameter parser (a comma here).

Arrays are much simpler to parse as depicted in the script 18.38.

Script 18.38: *Defining the JSON parser for array as represented in Figure 18.7*

```

PPJsonGrammar>>array
^ ${ asParser token trim ,
    elements optional ,
    $} asParser token trim

PPJsonGrammar>>elements
^ value separatedBy: $, asParser token trim
  
```

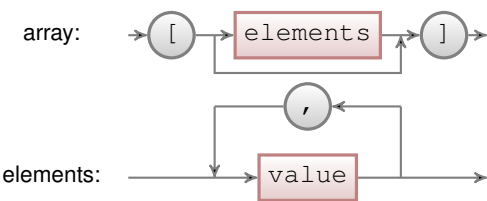


Figure 18.7: Syntax diagram representation for the JSON array parser defined in script 18.38

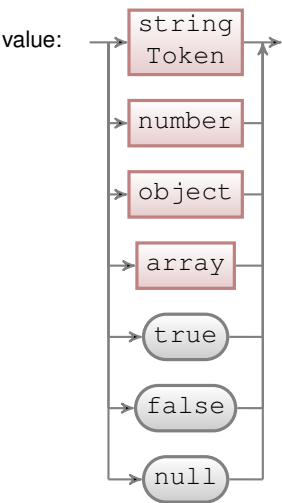


Figure 18.8: Syntax diagram representation for the JSON value parser defined in script 18.39

Parsing values

In JSON, a value is either a string, a number, an object, an array, a Boolean (true or false), or null. The value parser is defined as below and represented in Figure 18.8:

```
Script 18.39: Defining the JSON parser for value as represented in Figure 18.8
PPJsonGrammar>>value
^ stringToken / numberToken / object / array /
  trueToken / falseToken / nullToken
```

A string requires quite some work to parse. A string starts and end with double-quotes. What is inside these double-quotes is a sequence of characters. Any character can either be an escape character, an octal character, or a

normal character. An escape character is composed of a backslash immediately followed by a special character (e.g., '\n' to get a new line in the string). An octal character is composed of a backslash, immediately followed by the letter 'u', immediately followed by 4 hexadecimal digits. Finally, a normal character is any character except a double quote (used to end the string) and a backslash (used to introduce an escape character).

Script 18.40: *Defining the JSON parser for string as represented in Figure 18.9*

```
PPJsonGrammar>>stringToken
^ string token trim
PPJsonGrammar>>string
^ $" asParser , char star , $" asParser
PPJsonGrammar>>char
^ charEscape / charOctal / charNormal
PPJsonGrammar>>charEscape
^ $ \ asParser , (PPPredicateObjectParser anyOf: (String withAll: CharacterTable keys))
PPJsonGrammar>>charOctal
^ '\u' asParser , (#hex asParser min: 4 max: 4)
PPJsonGrammar>>charNormal
^ PPPredicateObjectParser anyExceptAnyOf: \"
```

Special characters allowed after a slash and their meanings are defined in the CharacterTable dictionary that we initialize in the initialize class method. Please note that initialize method on a class side is called when the class is loaded into the system. If you just created the initialize method class was loaded without the method. To execute it, you should evaluate PPJsonGrammar initialize in your workspace.

Script 18.41: *Defining the JSON special characters and their meaning*

```
PPJsonGrammar class>>initialize
CharacterTable := Dictionary new.
CharacterTable
  at: $ \ put: $ \;
  at: $ / put: $ /;
  at: $ " put: $ ";
  at: $ b put: Character backspace;
  at: $ f put: Character newPage;
  at: $ n put: Character lf;
  at: $ r put: Character cr;
  at: $ t put: Character tab
```

Parsing numbers is only slightly simpler as a number can be positive or negative and integral or decimal. Additionally, a decimal number can be expressed with a floating number syntax.

Script 18.42: *Defining the JSON parser for number as represented in Figure 18.10*

```
PPJsonGrammar>>numberToken
```

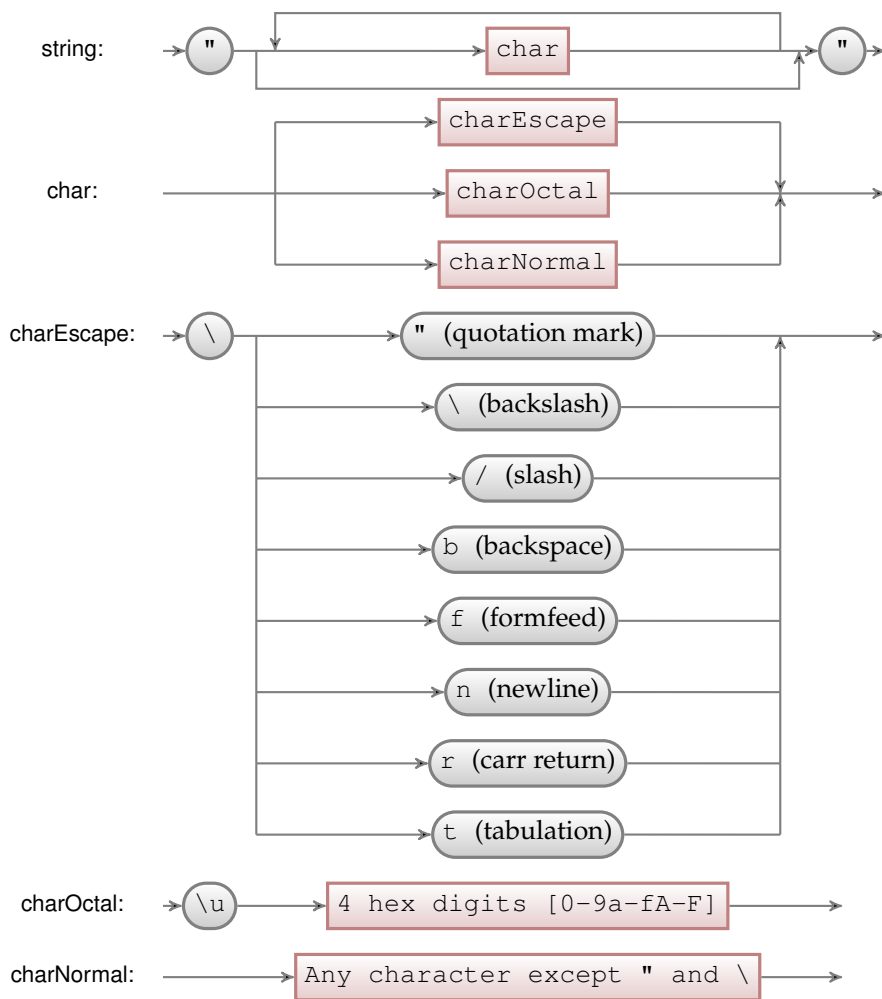


Figure 18.9: Syntax diagram representation for the JSON string parser defined in script 18.40

```

^ number token trim
PPJsonGrammar>>number
^ $- asParser optional ,
($0 asParser / #digit asParser plus) ,
($. asParser , #digit asParser plus) optional ,
(($e asParser / $E asParser) , ($- asParser / $+ asParser) optional , #digit asParser
plus) optional

```

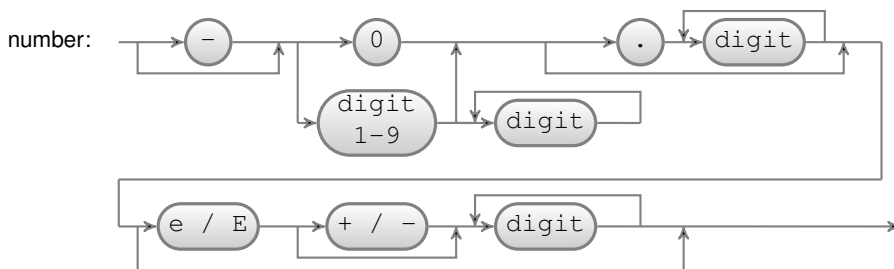


Figure 18.10: Syntax diagram representation for the JSON number parser defined in script 18.42

The attentive reader will have noticed a small difference between the syntax diagram in Figure 18.10 and the code in script 18.42. Numbers in JSON can not contain leading zeros: *i.e.*, strings such as "01" do not represent valid numbers. The syntax diagram makes that particularly explicit by allowing either a 0 or a digit between 1 and 9. In the above code, the rule is made implicit by relying on the fact that the parser combinator `$/` is ordered: the parser on the right of `$/` is only tried if the parser on the left fails: thus, `($0 asParser / #digit asParser plus)` defines numbers as being just a 0 or a sequence of digits not starting with 0.

The other parsers are fairly trivial:

Script 18.43: Defining missing JSON parsers

```

PPJsonGrammar>>>falseToken
^ 'false' asParser token trim
PPJsonGrammar>>>nullToken
^ 'null' asParser token trim
PPJsonGrammar>>>trueToken
^ 'true' asParser token trim

```

The only piece missing is the start parser.

Script 18.44: Defining the JSON start parser as being a value (Figure 18.8) with nothing following

```

PPJsonGrammar>>start
^ value end

```

18.5 PetitParser Browser

PetitParser is shipped with a powerful browser that can help to develop complex parsers. The PetitParser Browser provides graphical visualization, debugging support, refactoring support, and some other features discussed

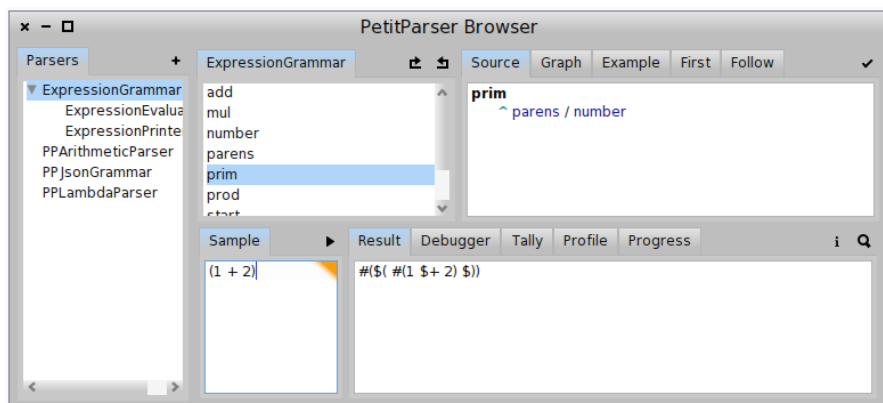


Figure 18.11: PetitParser Browser window.

later in this chapter. You will see that these features could be very useful while developing your own parser. Pay attention to have Glamour already loaded in your system. To load Glamour, see 10. Then to open the PetitParser simply evaluate this expression:

Script 18.45: *Opening PetitParser browser*

```
PPBrowser open.
```

PetitParser Browser overview

In Figure 18.11 you can see the PPBrowser window. The left panel, named **Parsers**, contains the list of all parsers in the system. You can see `ExpressionGrammar` and its subclasses as well as the `PPJsonGrammar` that we defined earlier in this chapter. Selecting one of the parsers in this pane activates the upper-right side of the browser. For each rule of the selected parser (e.g., `prim`) you can see 5 tabs related to the rule.

Source shows the source code of the rule. The code can be updated and saved in this window. Moreover, you can add a new rule simply by defining the new method name and body.

Graph shows the graphical representation of the rule. It is updated as the rule source is changed. You can see the `prim` visual representation in Figure 18.12.

Example shows an automatically generated example based on the definition of the rule (see Figure 18.13 for an example for the `prim` rule). In

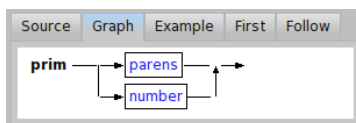


Figure 18.12: Graph visualization of the prim rule.

the top-right corner, the reload button generates a new example for the same rule (see Figure 18.14 for another automatically generated example of the prim rule, this time with a parenthesized expression).

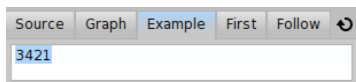


Figure 18.13: An automatically generated example of the prim rule. In this case, the prim example is a number.

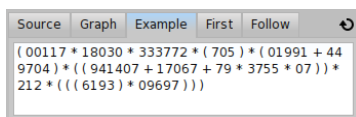


Figure 18.14: Another automatically generated example of the prim rule, after having clicked the reload button. In this case, the prim example is a parenthesized expression.

First shows set of terminal parsers that can be activated directly after the rule started. As you can see on Figure 18.15, the first set of prim is either digit or opening parenthesis '('. This means that once you start parsing prim the input should continue with either digit or '('.

One can use first set to double-check that the grammar is specified correctly. For example, if you see '+' in the first set of prim, there is something wrong with the definitions, because the prim rule was never ment to start with binary operator.

Terminal parser is a parser that does not delegate to any other parser. Therefore you don't see parens in prim first set because parens delegates to another parsers – trimming and sequence parsers (see script 18.46). You can see '(' which is first set of parens. The same states for number rule which creates action parser delegating to trimming parser delegating to flattening parser delegating to repeating parser delegating to

#digit parser (see script 18.46). The #digit parser is terminal parser and therefore you can see 'digit expected' in a first set. In general, computation of first set could be complex and therefore PPBrowser computes this information for us.

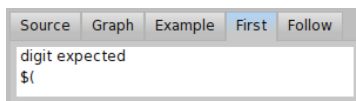


Figure 18.15: The first set of the prim rule.

Script 18.46: *prim rule in ExpressionGrammar*

```
ExpressionGrammar>>prim
  ^ parens / number

ExpressionGrammar>>parens
  ^ ${ asParser trim, term, $} asParser trim

ExpressionGrammar>>number
  ^ #digit asParser plus flatten trim ==> [:str | str asNumber ]
```

Follow shows set of terminal parsers that can be activated directly after the rule finished. As you can see on Figure 18.16, the follow set of prim is closing bracket character parser ')', star character parser '*', plus character parser '+' or epsilon parser (which states for empty string). In other words, once you finished parsing prim rule the input should continue with one of ')', '*', '+' characters or the input should be completely consumed.

One can use follow set to double-check that the grammar is specified correctly. For example if you see '(' in prim follow set, something is wrong in the definition of your grammar. The prim rule should be followed by binary operator or closing bracket, not by opening bracket.



Figure 18.16: The follow set of the prim rule.

In general, computation of follow could be even more complex than computation of first and therefore PPBrowser computes this information for us.

The lower-right side of the browser is related to a particular parsing input. You can specify an input sample by filling in the text area in the Sample tab. One may parse the input sample by clicking the play ► button or by pressing Cmd-s or Ctrl-s. You can then gain some insight on the parse result by inspecting the tabs on the bottom-right pane:

Result shows the result of parsing the input sample that can be inspected by clicking either the Inspect or Explore buttons. Figure 18.17 shows the result of parsing (1+2).

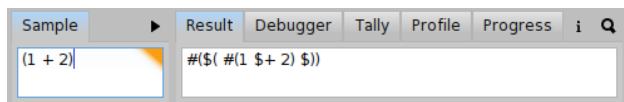


Figure 18.17: Result of parsing the (1+2) sample expression

Debugger shows a tree view of the steps that were performed during parsing. This is very useful if you don't know what exactly is happening during parsing. By selecting the step the subset of input is highlighted, so you can see which part of input was parsed by a particular step.

For example, you can inspect how the ExpressionGrammar works, what rules are called and in which order. This is depicted in Figure 18.18. The grey rules are rules that failed. This usually happens for choice parsers and you can see an example for the prod rule (the definition is in script 18.47). When parser was parsing $12 + 3 * 4$ term, the parser tried to parse mul rule as a first option in prod. But mul required star character '*' at position 2 which is not present, so that the mul failed and instead the prim with value 12 was parsed.

Script 18.47: prod rule in ExpressionGrammar

```
ExpressionGrammar>>prod
  ^ mul / prim
ExpressionGrammar>>mul
  ^ prim, $* asParser trim, prod
```

Compare what happens during parsing when we change from $12 + 3 * 4$ to $12 * 3 * 4$. What rules are applied now, which of them fails? The second debugger output is in Figure 18.19, but give it your own try.

Tally shows how many times a particular parser got called during the parsing. The percentage shows the number of calls to total number of calls ratio. This might be useful while optimizing performance of your parser (see Figure 18.20).

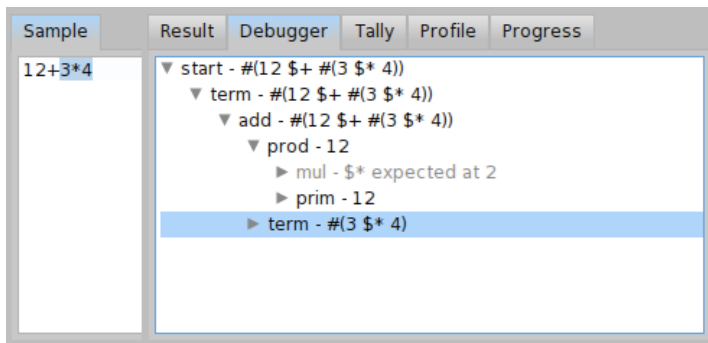


Figure 18.18: Debugger output of ExpressionGrammar for input $12 + 3 * 4$.

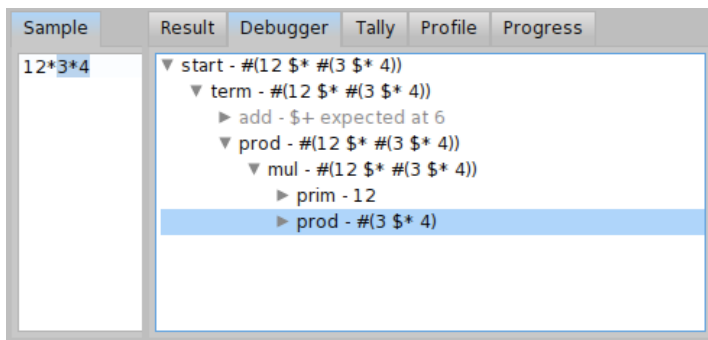


Figure 18.19: Debugger output of ExpressionGrammar for input $12 * 3 * 4$.

Profile shows how much time was spent in particular parser during parsing of the input. The percentage shows the ratio of time to total time. This might be useful while optimizing performance of your parser (see Figure 18.21).

Progress visually shows how a parser consumes input. The x-axis represents how many characters were read in the input sample, ranging from 0 (left margin) to the number of characters in the input (right margin). The y-axis represents time, ranging from the beginning of the parsing process (top margin) to its end (bottom margin). A line going from top-left to bottom-right (such as the one in Figure 18.22) shows that the parser completed its task by only reading each character of the input sample once. This is the best case scenario, parsing is linear in the length of the input: In another words, input of n characters is parsed in n steps.

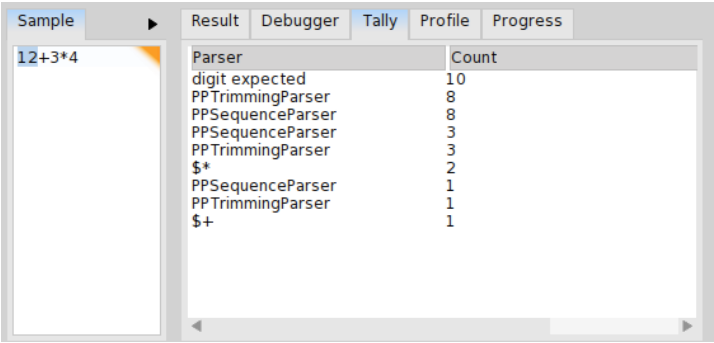


Figure 18.20: Tally of ExpressionGrammar for input 12 * 3 * 4.

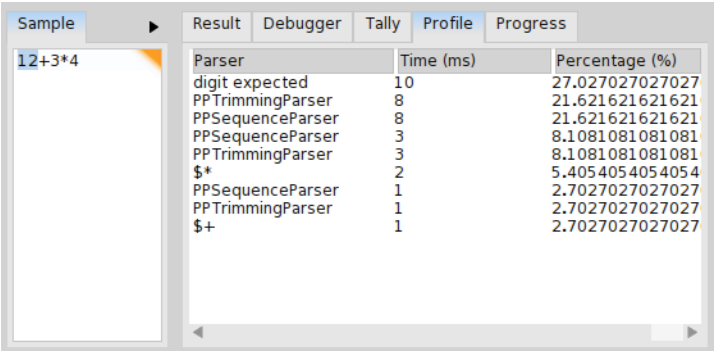


Figure 18.21: Profile of ExpressionGrammar for input 12 * 3 * 4.

When multiple lines are visible, it means that the parser had to go back to a previously read character in the input sample to try a different rule. This can be seen in Figure 18.23. In this example, the parser had to go back several times to correctly parse the whole input sample: all input was parsed in $n!$ steps which is very bad. If you see many backward jumps for a grammar, you should reconsider the order of choice parsers, restructure your grammar or use a memoized parser. We will have a detailed look on a backtracking issue in the following section.

Debugging example

As an exercise, we will try to improve a BacktrackingParser from script 18.48. The BacktrackingParser was designed to accept input corresponding to the regular expressions 'a*b' and 'a*c'. The parser gives us correct results, but there

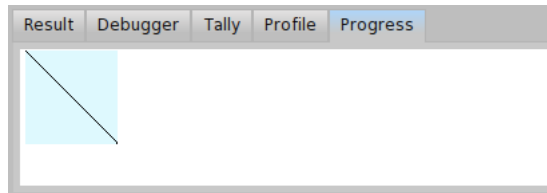


Figure 18.22: Progress of Petit Parser that parses input in linear amount of steps.

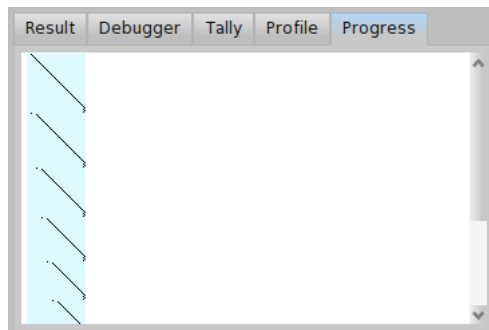


Figure 18.23: Progress of Petit Parser with a lot of backtracking.

is a problem with performance. The BacktrackingParser does too much backtracking.

Script 18.48: *A parser accepting 'a*b' and 'a*c' with too much backtracking.*

```
PPCompositeParser subclass: #BacktrackingParser
```

```
instanceVariableNames: 'ab ap c p'
```

```
classVariableNames: "
```

```
poolDictionaries: "
```

```
category: 'PetitTutorial'
```

```
BacktrackingParser>>ab
```

```
^ 'b' asParser /
```

```
('a' asParser, ab)
```

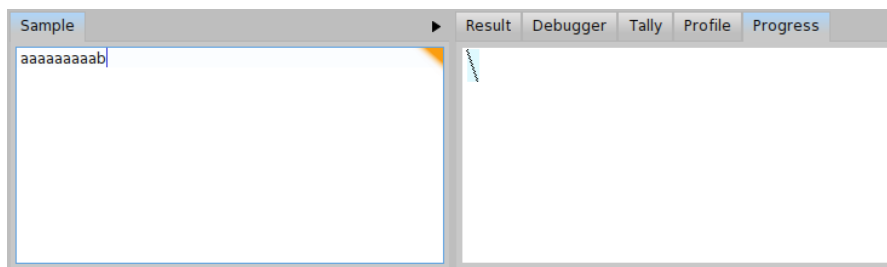
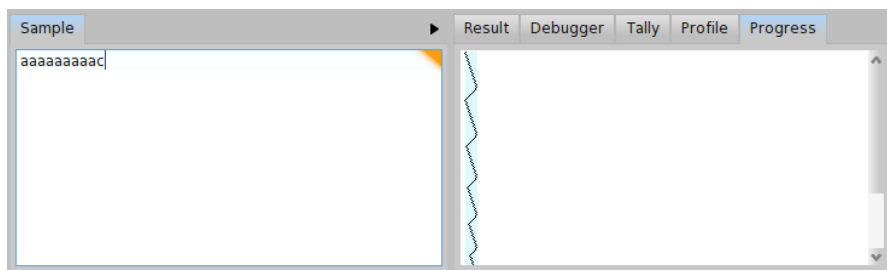
```
BacktrackingParser>>c
```

```
^ 'c' asParser
```

```
BacktrackingParser>>p
```

```
^ ab / ap / c
```

```
BacktrackingParser>>start
```

Figure 18.24: Progress of the BacktrackingParser for $input_b$.Figure 18.25: Progress of the BacktrackingParser for $input_c$.

```

^ p

BacktrackingParser>>ap
^ 'a' asParser, p

```

Let us get some overview to better understand, what is happening. First of all, try to parse $input_b = \text{'aaaaaaaaab'}$ and $input_c = \text{'aaaaaaaaac'}$. As we can see from progress depicted in Figure 18.24, the $input_b$ is parsed in more or less linear time and there is no backtracking. But the progress depicted in Figure 18.25 looks bad. The $input_c$ is parsed with a lot of backtracking and in much more time. We can even compare the tally output for both inputs $input_b$ and $input_c$ (see Figure 18.26 and Figure 18.27). In case of $input_b$, the total invocation count of the parser b is 19 and invocation count of the parser a is 9. It is much less than 110 invocations for the parser b and 55 invocations for the parser a in case of $input_c$.

We can see there is some problem with $input_c$. If we still don't know what is the problem, the debugger window might give us more hints. Let us have a look at the debugger window for $input_b$ as depicted in Figure 18.28. We can see that in each step, one 'a' is consumed and the parser ab is invoked until it

Parser	Count
'b'	19
'a'	9

Figure 18.26: Tally output of the BacktrackingParser for *input_b*.

Parser	Count
'b'	110
'a'	35
PPSequenceParser	35
'a'	11
PPSequenceParser	1
'c'	1

Figure 18.27: Tally output of the BacktrackingParser for *input_c*.

reaches the 'b'. The debugger window for *input_c* as depicted in Figure 18.29 looks much different. There is a progress within the `p -> ab -> ap -> p` loop but the parser `ab` fails in each repetition of the loop. Since the parser `ab` fails after having read all the string to the end and seen 'c' instead of 'b', we have localized the cause of the backtracking. We know the problem now, so what can we do? We may try to update BacktrackingParser so that the 'a*c' strings are parsed in a similar way as the 'a*b' strings. You can see such a modification in script 18.49.

Script 18.49: *A slightly better parser accepting 'a*b' and 'a*c'.*

```
PPCompositeParser subclass: #BacktrackingParser
```

```
  instanceVariableNames: 'ab ac'
```

```
  classVariableNames: "
```

```
  poolDictionaries: "
```

```
  category: 'PetitTutorial'
```

```
BacktrackingParser>>ab
```

```
  ^ 'b' asParser /
```

```
  ('a' asParser, ab)
```

```
BacktrackingParser>>ac
```

```
  ^ 'b' asParser /
```

```
  ('c' asParser, ab)
```

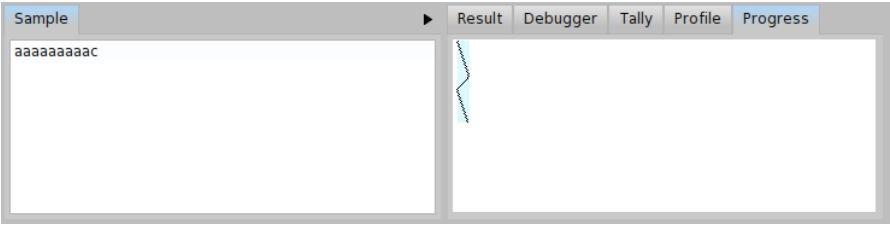



Figure 18.30: Progress of BacktrackingParser for *input_c* after the first update.

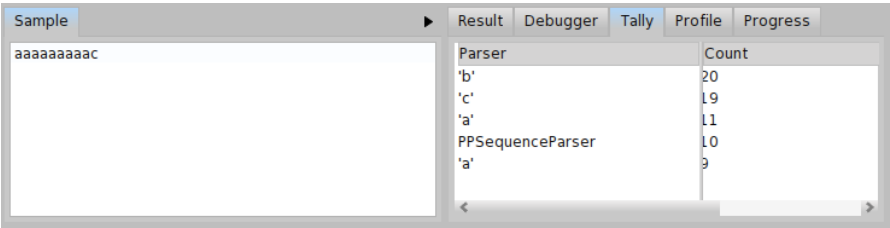


Figure 18.31: Tally of BacktrackingParser for *input_c* after the first update.

'c' at the very end? You can see such a modification of the BacktrackingParser in script 18.50. In that case, we can see the progress without any backtracking even for *input_c* as depicted in Figure 18.32.

On the other hand, the number of parser invocations for *input_b* increased by 18 (the Table 18.4 summarizes the total number of invocations for each version of the BacktrackingParser). It is up to the developer to decide which grammar is more suitable for his needs. It is better to use the second improved version in case 'a*b' and 'a*c' occur with the same probability in the input. If we expect more 'a*b' strings in the input, the first version is better.

Script 18.50: *An even better parser accepting 'a*b' and 'a*c'.*

```
PPCompositeParser subclass: #BacktrackingParser
  instanceVariableNames: 'abc'
  classVariableNames: ""
  poolDictionaries: ""
  category: 'PetitTutorial'
```

```
BacktrackingParser>>abc
^ ('b' asParser / 'c' asParser) /
  ('a' asParser, abc)
```

```
BacktrackingParser>>start
^ abc
```

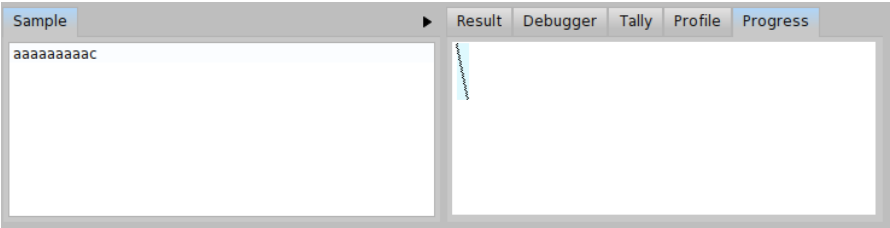


Figure 18.32: Progress of the BacktrackingParser after the second update for $input_c$.

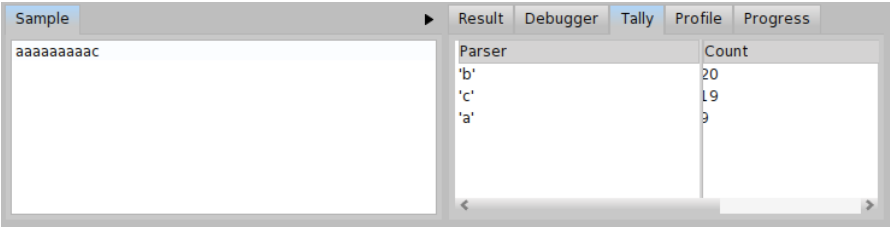


Figure 18.33: Tally of the BacktrackingParser after the second update for $input_c$.

Version	# of invocations	
	$input_b$	$input_c$
Original	28	233
First improvement	28	70
Second improvement	46	48

Table 18.4: Number of parser invocations for $input_b$ and $input_c$ depending on the version of BacktrackingParser.

18.6 Packrat Parsers

In the beginning of the chapter, we have mentioned four parser methodologies, one of them was *Packrat Parsers*. We claimed that packrat parsing gives linear parse times. But in the debugging example we saw that original version of the BacktrackingParser parsed $input_c$ of length 10 in 233 steps. And if you try to parse $longinput_c = 'aaaaaaaaaaaaaaaaaaaaaac'$ (length 20), you will see that the original parser needs 969 steps. Indeed, the progress is not linear.

The PetitParser framework does not use packrat parsing by default. You need to send the memoized message to enable packrat parsing. The memo-

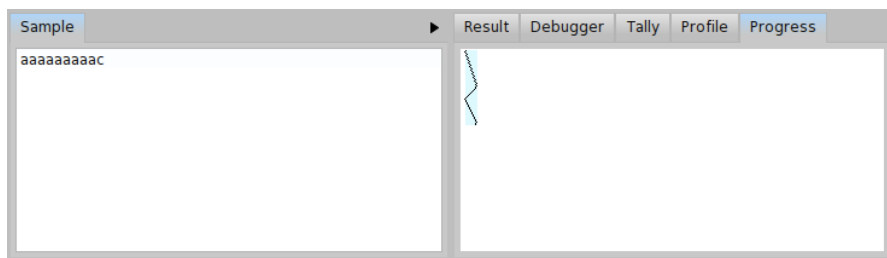


Figure 18.34: Progress of the memoized version of the BacktrackingParser.

ized parser ensures that the parsing for the particular position in an input and the particular parser will be performed only once and the result will be remembered in a dictionary for a future use. The second time the parser wants to parse the input, the result will be looked up in the dictionary. This way, a lot of unnecessary parsing can be avoided. The disadvantage is that PetitParser needs much more memory to remember all the results of all the possible parsers at all the possible positions.

To give you an example with a packrat parser, let us return back to the BacktrackingParser once again (see script 18.48). As we have analyzed before, the problem was in the parser `ab` that constantly failed in the `p -> ab -> ap -> p` loop. Now we can do the trick and memoize the parser `ab` by updating the method `ab` as in script 18.51. When the memoization is applied, we get the progress as in Figure 18.34 with the total number of 63 invocations for *input_c* and the 129 invocations for *longinput_c*. With the minor modification of BacktrackingParser we got a linear parsing time (related to the length of the input) with a factor around 6.

Script 18.51: *Memoized version of the parser ab.*

```
BacktrackingParser>>ab
^ ( 'b' asParser /
  ('a' asParser, ab)
) memoized
```

18.7 Chapter summary

This concludes our tutorial of PetitParser. We have reviewed the following points:

- A parser is a composition of multiple smaller parsers combined with combinators.

- To parse a string, use the method `parse`.
- To know if a string matches a grammar, use the method `matches`.
- The method `flatten` returns a `String` from the result of the parsing.
- The method `==>` performs the transformation given in the block given in parameter.
- Compose parsers (and create a grammar) by subclassing `PPCompositeParser`.
- Test your parser by subclassing `PPCompositeParserTest`.

For a more extensive view of PetitParser, its concepts and implementation, the Moose book⁴ and Lukas Renggli's PhD⁵ have both a dedicated chapter.

⁴<http://www.themoosebook.org/book/internals/petit-parser>

⁵<http://scg.unibe.ch/archive/phd/renggli-phd.pdf>